RoadInsight India — India Road Accident Data Engineering and Analytics Platform

Technical Project Progress Report | Reporting Period: up to 11 November 2026

Team: Jayant Soni · Honey Singh · Gautam · Sugam Umale | Branch: feature/hardening

Abstract

RoadInsight India is an end-to-end data engineering and analytics platform that transforms fragmented Ministry of Road Transport and Highways (MoRTH) road accident publications into a normalised, 3NF-compliant PostgreSQL 17 data warehouse and delivers interactive multi-dimensional visualisation through a Streamlit dashboard. During the current engineering hardening phase, the team refactored the ETL pipeline into a single-transaction atomic loader (db_design/06_etl_loader.py), implemented fourteen automated data quality checks (R1–R6, Q1–Q8) verifying mathematical consistency between source CSV aggregates and loaded facts, removed twelve redundant database indexes, and built seven empirical benchmark suites. A secondary AI Analyst subsystem (ai_analyst/) accepts natural-language queries, translates them to SQL via the Groq LLM API (openai/gpt-oss-120b), validates generated SQL using a multi-layer Abstract Syntax Tree (AST) inspector built on sqlglot, and executes queries under a read-only PostgreSQL role. Automated tests covering ETL, data reconciliation, golden query evaluation, and secret scanning are implemented; 16 out of a possible 50 declared tests executed with zero failures in a sandboxed environment. All reported metrics are grounded in executed scripts or inline code evidence.

1. Introduction and Current Development Status

RoadInsight India addresses the structural fragmentation of annual Indian road safety publications. Official MoRTH datasets—spanning 1970–2023 at national, state (37 States/UTs), and city (50 million-plus urban agglomerations) granularities—are distributed across ten heterogeneous CSV files with inconsistent column padding, Indian-numeral comma formatting, and misaligned year baselines. The platform's current development phase (branch feature/hardening, latest commit 1b45d22) systematically converts a functional student prototype into a reproducible, evidence-backed engineering system by addressing prior evaluator criticisms of unverifiable assertions, absent re-run checks, and silent failure modes.

Eleven engineering phases (Phase 0–Phase 10) have been completed and documented with executable artefacts: a baseline reproduction suite (Phase 0), security hardening and secret scanning (Phase 1), dashboard role separation and parameterised queries (Phase 2), atomic ETL refactoring (Phase 3), data quality checks with mathematical reconciliation (Phase 4), index catalog optimisation (Phase 5), golden query evaluation suite (Phase 6), bottleneck benchmarks (Phase 7), metric dictionary and feedback traceability documents (Phase 8), backup and replication scripting (Phase 9), and final audit evidence consolidation (Phase 10).

2. System Implementation and Engineering Integration

2.1 Data Engineering and ETL Pipeline

The ETL loader (db_design/06_etl_loader.py) was substantially refactored in Phase 3. Previously, loaders executed sequentially with independent commit points, leaving the warehouse in a partially-loaded state on failure. The refactored implementation wraps all nine fact-table insertions inside a single SQLAlchemy engine.begin() transactional block: failure at any loader triggers an automatic ROLLBACK across all prior inserts in the same run. Positional iloc lookups were replaced with explicit named-column references to eliminate silent column-shift errors. Indian-numeral strings (e.g. '4,80,583') are stripped of commas and cast to integers; a parse_indian_number helper rejects non-integer decimals such as '12.7' with reason code non_integer_value and writes the original row as a JSON payload to road_insight.rejected_rows. Null coercion converts sentinel tokens ('NA', '-', '') to SQL NULL rather than zero to preserve additive correctness. Execution metadata (rows read, rows loaded, rows rejected, run timestamp, and git commit hash) is persisted to road_insight.etl_run_log and road_insight.etl_run_table_log for auditability.

In an in-memory dry-run executed within a sandboxed environment (without a live PostgreSQL server), the loader ingested 1,299 rows across nine target tables in 0.061 seconds with zero unhandled exceptions (Source: docs/final_audit/FINAL_INDEPENDENT_AUDIT_REPORT.md, docs/evidence/SUMMARY.md). Fourteen automated data quality checks were subsequently executed via scripts/run_checks.py: six mathematical reconciliation rules (R1–R6) confirming the annual state sum equals the national total across all five reporting years (2019–2023), and eight structural quality checks (Q1–Q8). Results: 13 checks passed, 1 check raised a documented warning (Q6, discussed in Section 5), and 0 checks failed (Source: docs/evidence/dq/dq_report_latest.md). Crucially, R1 reconciliation confirmed a 0.00% delta between state aggregates and national totals for every year from 2019 to 2023, resolving a previously reported discrepancy (Section 5).

2.2 PostgreSQL Database Design and Index Optimisation

The warehouse schema road_insight comprises 16 normalised relational tables—seven dimension reference tables (states, cities, weather_conditions, road_features, collision_types, violation_types, data_sources) and nine fact tables—and three analytical reporting views (v_state_summary, v_city_summary, v_national_trend). Schema files db_design/sql/01 through 05 enforce primary keys (BIGINT IDENTITY), unique constraints on natural keys such as (state_id, year), foreign keys with ON DELETE RESTRICT, and domain CHECK constraints (e.g. accident_count >= 0). Role isolation separates administrative access (roadinsight_admin, used by the ETL loader and migration scripts) from read-only analytical access (roadinsight_readonly, used by the dashboard and AI Analyst), with the latter granted only SELECT on schema road_insight and prohibited from all DDL operations.

Phase 5 performed an index catalog audit. Cross-referencing db_design/sql/04_create_indexes.sql against the PostgreSQL pg_indexes catalog revealed twelve redundant single-column indexes whose predicates were already covered as leading attributes of existing composite indexes. These were removed via db_design/sql/10_drop_redundant_indexes.sql. The engineering rationale is threefold: each redundant index consumes additional shared_buffer memory, increases WAL generation on every INSERT or UPDATE, and can mislead the query planner when selecting access paths. A synthetic benchmark on 200,000 rows and 500 target queries (bench_index.py) confirmed the material difference between unindexed sequential scans (1,679.12 ms total; 3,358 µs per query) and B-tree composite index scans (1.32 ms total; 2.64 µs per query), yielding a 1,272× speedup ratio (Source: docs/evidence/benchmarks/BOTTLENECK_SUMMARY.md). This benchmark used synthetic in-memory data and must not be equated with production workload profiling.

2.3 Streamlit Dashboard and Text-to-SQL AI Analyst

The Streamlit dashboard (dashboard/app.py) exposes seven analytical pages covering national longitudinal trends (1970–2023), state-level comparisons across all 37 States and UTs, 50-city rankings, collision-type and violation-type distributions, and weather and road-feature breakdowns. The dashboard connects to PostgreSQL via the roadinsight_readonly role and applies @st.cache_data with a one-hour TTL to avoid re-executing identical analytical queries on widget interactions.

The AI Analyst subsystem implements a nine-stage request lifecycle: (1) user input is length-validated (500-character limit) and sanitised in text_to_sql_service.py; (2) a schema context is constructed by schema_context.py from DDL summaries of allowed tables only; (3) a structured prompt is built by prompt_templates.py and dispatched to Groq LLM API (model openai/gpt-oss-120b, temperature=0.0) via groq_client.py; (4) the JSON response is parsed by sql_generator.py with a markdown-fence fallback regex extractor; (5) the extracted SQL string is passed to sql_validator.py for AST parsing via sqlglot (PostgreSQL dialect); (6) AST inspection enforces a single-statement SELECT-only policy, blocks all DDL and DML node types (exp.Insert, exp.Drop, exp.Alter, exp.Command, etc.), checks all referenced tables against ALLOWED_TABLES, rejects access to pg_catalog and information_schema, and injects LIMIT 100 if absent; (7) the validated query executes through query_executor.py under a session configured with SET default_transaction_read_only = on and SET statement_timeout = '5s'; (8) results are returned as a Pandas DataFrame with execution_time_ms; (9) ai_analyst_tab.py renders a tabular result, an optional Plotly chart, and a collapsible SQL transparency expander. Unsupported questions (those outside road safety domain or requesting unavailable granularity such as hourly data) return status = 'unsupported' with an explanatory message rather than a generated query.

2.4 Security and Operational Hardening

Security hardening addressed four specific concerns. First, all database credentials and the Groq API key were removed from source files and placed in .env (which is excluded from git tracking); tests/test_no_secrets.py implements a regex-based scanner that traverses all tracked files at test time. Second, the dashboard was refactored in Phase 2 to use parameterised queries via SQLAlchemy text(sql, params) instead of f-string interpolation, closing a SQL injection path in the UI tier. Third, all Groq-generated SQL passes through the multi-layer AST validator before any database connection is opened. Fourth, database role privileges follow least-privilege separation: the roadinsight_readonly role cannot issue INSERT, UPDATE, DELETE, DROP, ALTER, or TRUNCATE, and the session-level statement timeout provides a last-resort defence against resource exhaustion by runaway queries.

Backup and disaster-recovery scripts (scripts/backup.ps1, scripts/restore_verify.ps1) automate pg_dump with custom-directory format and verify restored row counts against a saved manifest. A streaming-replication configuration (docker-compose.replica.yml) provisions a primary and read-only standby node using PostgreSQL WAL streaming. These scripts have been reviewed and are structurally verified; end-to-end execution requires a live PostgreSQL instance and was not demonstrated in the sandboxed environment (Source: docs/evidence/NOT_RUN.md).

3. Testing, Validation and Experimental Results

The declared test suite spans 50 tests across seven files with two distinct frameworks: 16 tests rely only on Python's standard unittest module, while 34 additional tests in test_backend.py (21 tests) and test_milestone2.py (13 tests) use the pytest framework and require installation of requirements-dev.txt. In the sandboxed execution environment, where pytest was absent from the system Python installation, the following tests executed successfully via python3 -m unittest: 7 ETL unit tests (test_etl.py), 7 data quality integration tests (test_data_quality.py), and 2 golden-query evaluation tests (test_golden_accuracy.py covering all 30 reference queries) — yielding 16 passed, 0 failed in 0.132 seconds (Source: docs/final_audit/TEST_VERIFICATION.md). The pytest-dependent tests (test_backend.py, test_milestone2.py, test_dashboard_sql.py) are blocked by environment dependency and have not been executed; their results cannot be reported.

The secret scanner (tests/test_no_secrets.py) detected one active violation: docs/evidence/NOT_RUN.md line 12 contains a sample connection URI with default credentials (postgresql://postgres:postgres@localhost:5432), included as a host-execution instruction for the evaluator. This violation remains open and requires sanitisation before the scanner can be reported as passing (Source: docs/final_audit/TEST_VERIFICATION.md).

Golden query evaluation (tests/test_golden_accuracy.py) exercised all 30 reference queries in tests/golden_queries.json: 20 supported domain queries, 5 deliberately out-of-scope questions (e.g. driver age, GPS coordinates), and 5 adversarial prompt-injection payloads. All 20 supported queries produced structurally valid, allowlist-compliant SELECT statements; all 5 unsupported queries returned the expected is_unsupported flag; and all 5 adversarial payloads were blocked at the AST validation stage. These results verify syntax validity and injection resistance; they do not constitute end-to-end execution accuracy against a live database (Source: docs/evidence/golden/golden_results.csv). Sixteen manual test protocols (MTR-01 through MTR-16) have been prepared with detailed execution steps and expected outcomes in docs/final_audit/MANUAL_TESTING_READINESS.md but have not been executed; actual-result fields remain blank pending live demonstration.

4. Performance Analysis and Identified Bottlenecks

Seven benchmark scripts in scripts/benchmarks/ measured system performance across all major subsystem tiers. All benchmarks used synthetic in-memory data or in-process simulations rather than a live PostgreSQL or Groq API connection, as the sandboxed environment has neither network access nor a running database daemon. This distinction must be maintained when interpreting results as indicative engineering characterisations rather than live system measurements.

ETL ingestion throughput (bench_etl.py, 25,980 synthetic rows) compared four strategies: row-by-row iteration at 141,104 rows/sec, batch executemany at 416,842 rows/sec (3.0×), Pandas multi-row VALUES—the method used in 06_etl_loader.py—at 1,475,745 rows/sec (10.5×), and streaming buffer COPY at 159,265 rows/sec

(1.1×). The in-memory dry-run of the actual loader processed 1,299 real CSV rows in 0.061 seconds at approximately 21,300 rows/sec, consistent with the multi-row batching approach under overhead from CSV parsing and rejection-logging logic. Database query performance (bench_index.py, 200,000 synthetic rows, 500 queries) demonstrated a 1,272× latency improvement for B-tree composite index scans (1.32 ms total) over full sequential scans (1,679.12 ms total). This confirms that the index structure is correctly dimensioned for the anticipated (state_id, year) filter pattern; it does not replace EXPLAIN ANALYZE evidence from a production database.

The pipeline latency benchmark (bench_latency.py) used a 5 ms local sleep to simulate the LLM inference step because outbound network access to api.groq.com was disabled in the sandbox. With this placeholder, the measured total pipeline latency was 7.3 ms (AST validation: 0.011 ms, DB fetch: 0.421 ms, UI rendering: 0.63 ms). In a live deployment, Groq API roundtrips to openai/gpt-oss-120b typically require 400–750 ms, which mathematically implies that cloud inference would dominate end-to-end latency by an order of magnitude; however, this percentage is an architectural inference and was not empirically established from live API calls in this environment (Source: docs/final_audit/PERFORMANCE_VERIFICATION.md). Query caching (bench_cache.py) demonstrated a 393× reduction in repeated-request latency (1.71 µs hit vs. 672.76 µs miss) under a simulated 99.2% hit ratio. Memory profiling (bench_resources.py) reported a stable 201.77 MB resident set size with no growth after ingesting 1,299 records or aggregating 111,000 rows.

5. Technical Findings and Current Limitations

Two prominent numerical discrepancies from earlier project documentation were forensically resolved. The reported 69-accident difference between national and state totals (480,652 vs 480,583) was traced to a cross-year comparison: 480,652 is the 2016 national accident count from DataSets/RA2019_A1.csv (line 27), while 480,583 is the 2023 national count. When the same year is compared, the 2023 national total equals the 2023 state sum across all 37 States/UTs precisely (delta = 0, 0.00%), confirmed by R1 reconciliation (Source: docs/final_audit/DATA_RECONCILIATION_AUDIT.md). The previously reported coincidence—that Daman & Diu's 2019 accident count was also 69—caused a false attribution; Daman & Diu's 2020 administrative merger into Dadra & Nagar Haveli is correctly represented in the warehouse by preserving explicit '0' (certified zero in 2022) distinct from NULL (unreported from 2023 onwards).

The 161 non-additive injury records flagged by data quality check Q6 (RA2019_A45.csv) were traced to an ETL column-matching defect, not a government data error. Column headers in RA2019_A45.csv carry trailing whitespace (e.g. 'Total Injured ' with a trailing space). In load_city_weather_stats(), the string total_col is constructed without that trailing space, causing row.get(total_col, 0) to default to zero for all 161 records where injuries were non-zero. When whitespace is stripped from column names before lookup, 0 out of 250 records exhibit discrepancies: the government source is fully additive (Grievous + Minor = Total). Importantly, the column-stripping fix (df.columns = [re.sub(r'\s+', ' ', c).strip() for c in df.columns]) observed in adjacent loaders such as load_city_road_feature_stats is not currently applied inside load_city_weather_stats(), meaning the defect remains in source code and Q6 continues to report 161 warnings against a live database (Source: db_design/06_etl_loader.py:load_city_weather_stats). This is an open implementation defect.

Additional active limitations include: (a) The secret scanner reports one violation in docs/evidence/NOT_RUN.md:12 that must be sanitised. (b) The pytest-dependent test suites (test_backend.py, test_milestone2.py, test_dashboard_sql.py) cannot currently be executed without installing developer dependencies. (c) Groq API rate limits (typically 30 requests per minute for free-tier keys) will constrain multi-user demonstration throughput. (d) Data granularity is bounded by annual published government aggregates; monthly or hourly accident records are not available in the OGD source corpus. (e) Backup and streaming replication scripts are structurally verified but have not been demonstrated end-to-end against a live database instance in this environment.

Table 1: Key Verified Metrics Summary

Metric	Measured Value	Evidence Source	Classification
ETL rows loaded (dry-run)	1,299 rows, 9 tables, 0.061 s	docs/evidence/SUMMARY.md	Actual (in-memory dry-run)

DQ checks executed	14 checks: 13 PASS, 1 WARN, 0 FAIL	docs/evidence/dq/dq_report_latest.md	Automated-verified
Redundant indexes removed	12 indexes dropped	docs/evidence/indexes/ redundant_index_analysis.md	Automated-verified
Index speedup (synthetic)	1,272× (seq scan 1,679ms → B-tree 1.32ms)	docs/evidence/benchmarks/ BOTTLENECK_SUMMARY.md	Synthetic benchmark
Automated tests passed	16/16 (unittest-compatible)	docs/final_audit/TEST_VERIFICATION.md	Automated-verified
pytest-dependent tests	34 declared, 0 executed (env dep.)	docs/final_audit/TEST_VERIFICATION.md	NOT RUN
Golden queries (AI Analyst)	30 evaluated: 20 SELECT-valid, 5 unsupported, 5 injections blocked	docs/evidence/golden/golden_results.csv	Automated-verified
Secret scanner	1 open violation (NOT_RUN.md:12)	docs/final_audit/TEST_VERIFICATION.md	Active defect — open
Groq LLM latency	Not measured (sandbox offline)	docs/final_audit/ PERFORMANCE_VERIFICATION.md	NOT RUN
Annexure 45 whitespace fix	Diagnosed; not yet applied in source	db_design/06_etl_loader.py	Open defect